



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251218
Nama Lengkap	Agatha Sabda Alethea Prabawa
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Pengenalan dan Mendefinisikan Set

Set adalah salah satu tipe data yang tersedia dalam bahasa pemrograman Python yang digunakan untuk menyimpan sekumpulan data yang bersifat unik. Dalam konteks matematika, set juga dikenal dengan istilah himpunan. Setiap nilai yang tersimpan di dalam set disebut sebagai anggota atau member. Keunikan set terletak pada kemampuannya untuk secara otomatis memastikan bahwa tidak ada anggota yang bernilai sama atau duplikat di dalamnya.

Set dalam Python memiliki beberapa sifat penting yang perlu dipahami. Pertama, setiap anggota yang dimasukkan ke dalam set harus bersifat immutable, artinya nilainya tidak dapat diubah setelah dibuat. Tipe data yang termasuk immutable dan dapat dijadikan anggota set antara lain integer, float, string, dan tuple. Sebaliknya, tipe data yang bersifat mutable seperti list dan dictionary tidak dapat dijadikan anggota set.

Kedua, meskipun anggotanya harus immutable, set itu sendiri bersifat mutable. Hal ini berarti kita dapat menambah atau mengurangi anggota di dalam set kapan saja setelah set dibuat. Namun karena set bersifat mutable, set tidak dapat dijadikan anggota dari set lain.

Ketiga, set tidak mengenal konsep urutan atau posisi. Berbeda dengan list atau tuple yang menyimpan data secara berurutan berdasarkan indeks, anggota-anggota dalam set tidak memiliki posisi yang tetap. Akibatnya, setiap kali isi set ditampilkan, urutannya bisa berbeda-beda dan tidak bisa diprediksi.

Untuk membuat sebuah set, Python menyediakan dua cara. Cara pertama adalah menggunakan notasi kurung kurawal {} dengan menuliskan anggota-anggota set di dalamnya yang dipisahkan oleh tanda koma. Cara kedua adalah menggunakan fungsi bawaan set(). Perlu diperhatikan bahwa untuk membuat set yang kosong, kita tidak bisa menggunakan notasi {}

karena notasi tersebut akan diinterpretasikan oleh Python sebagai dictionary kosong, bukan set kosong. Untuk membuat set kosong, wajib menggunakan fungsi `set()` tanpa argumen.

Pengaksesan Set

Karena set tidak memiliki indeks, anggota-anggota di dalamnya tidak dapat diakses secara langsung menggunakan indeks seperti pada list atau tuple. Cara yang umum digunakan untuk mengakses seluruh isi set adalah melalui perulangan `for`, yang akan menelusuri setiap anggota set satu per satu. Selain itu, fungsi `len()` dapat digunakan untuk mengetahui jumlah anggota yang ada di dalam set.

Satu hal yang perlu diperhatikan adalah bahwa urutan anggota yang ditampilkan saat iterasi tidak selalu sama dengan urutan saat set pertama kali dibuat. Hal ini merupakan perilaku normal dari set karena memang tidak ada konsep posisi atau urutan di dalamnya.

Karena set bersifat mutable, kita dapat menambah anggota baru ke dalam set menggunakan method `add()`. Salah satu keunggulan method `add()` adalah kemampuannya melakukan pengecekan duplikasi secara otomatis. Jika anggota yang ingin ditambahkan sudah ada di dalam set, maka pemanggilan `add()` tidak akan mengubah isi set sama sekali. Dengan kata lain, set secara otomatis menjaga keunikan setiap anggotanya tanpa perlu pengecekan manual dari programmer.

Untuk menghapus anggota dari set, Python menyediakan empat method yang berbeda dengan karakteristik masing-masing:

- `discard()` : menghapus anggota yang disebutkan dari set. Jika anggota yang dimaksud tidak ditemukan di dalam set, method ini tidak akan menghasilkan error dan set tetap tidak berubah.
- `remove()` : juga menghapus anggota yang disebutkan, namun berbeda dengan `discard()`, method ini akan menghasilkan pesan error jika anggota yang ingin dihapus ternyata tidak ada di dalam set.

- `pop()` : mengambil dan mengeluarkan salah satu anggota secara acak dari set. Method ini berguna ketika kita ingin memproses isi set satu per satu tanpa mempedulikan urutan. Jika set dalam kondisi kosong, method ini akan menghasilkan error.
- `clear()` : menghapus seluruh anggota yang ada di dalam set sekaligus sehingga set menjadi kosong. Method ini tidak menghasilkan error meskipun set sudah dalam kondisi kosong.

Selain operasi penambahan dan penghapusan, perlu dipahami pula cara mengubah nilai anggota dalam set. Berbeda dengan list, set tidak menyediakan mekanisme untuk mengubah nilai anggota secara langsung karena anggota set harus bersifat immutable. Cara yang dapat dilakukan adalah dengan operasi penggantian, yaitu menghapus anggota lama menggunakan `remove()` atau `discard()`, kemudian menambahkan anggota baru dengan nilai yang diinginkan menggunakan `add()`. Perlu diingat pula bahwa setiap kali terjadi penambahan atau penghapusan anggota, urutan tampilan isi set bisa berubah karena set tidak mengenal urutan yang tetap.

Operasi-Operasi pada Set

Set dalam Python mendukung berbagai operasi layaknya operasi himpunan dalam matematika. Terdapat empat operasi utama yang dapat dilakukan pada set, yaitu union, intersection, difference, dan symmetric difference. Setiap operasi tersebut dapat dilakukan menggunakan dua cara, yaitu menggunakan operator simbolik maupun menggunakan method bawaan Python.

Operator Union

Operasi union digunakan untuk menggabungkan dua set menjadi satu set baru yang memuat seluruh anggota dari kedua set tersebut. Dalam Python, operasi ini dapat dilakukan menggunakan operator `|` atau menggunakan method `union()`. Apabila terdapat anggota yang sama di kedua set (duplikasi), anggota tersebut hanya akan muncul satu kali saja di dalam set hasil karena sifat unik dari set. Sebagai contoh, jika set pertama berisi merek-merek handphone dan set kedua berisi merek-merek AC, penggabungan keduanya akan menghasilkan set baru yang

memuat semua merek dari kedua set, dengan merek yang muncul di keduanya hanya dihitung sekali.

Operator Intersection

Operasi intersection digunakan untuk mendapatkan irisan dari dua set, yaitu kumpulan anggota yang terdapat di kedua set secara bersamaan. Dalam Python, operasi ini dapat dilakukan menggunakan operator `&` atau menggunakan method `intersection()`. Hasil dari operasi intersection hanya berisi anggota-anggota yang menjadi milik kedua set sekaligus. Sebagai contoh, jika set pertama berisi nama-nama orang yang bisa renang dan set kedua berisi nama-nama orang yang bisa tenis, maka hasil intersection-nya adalah nama-nama orang yang mampu melakukan keduanya.

Operator Difference

Operasi difference digunakan untuk mendapatkan selisih antara dua set, yaitu kumpulan anggota yang hanya ada di set pertama tetapi tidak ada di set kedua. Dalam Python, operasi ini dilakukan menggunakan operator `-` atau method `difference()`. Penting untuk diperhatikan bahwa operasi difference bersifat tidak komutatif, artinya $A - B$ tidak sama dengan $B - A$. Sebagai contoh, jika terdapat set orang yang bisa berbahasa Inggris dan set orang yang bisa berbahasa Korea, maka operasi `Korean - English` akan menghasilkan orang yang hanya bisa bahasa Korea saja, sementara `English - Korean` akan menghasilkan orang yang hanya bisa bahasa Inggris saja.

Operator Symmetric Difference

Operasi symmetric difference menghasilkan set baru yang berisi anggota-anggota yang hanya terdapat di salah satu set saja, tidak termasuk anggota yang menjadi irisan (terdapat di keduanya). Dengan kata lain, symmetric difference adalah gabungan kedua set dikurangi irisannya. Dalam Python, operasi ini dapat dilakukan menggunakan operator `^` atau method `symmetric_difference()`. Sebagai contoh penggunaan, jika kita ingin mengetahui siapa saja yang hanya bisa berbicara dalam satu bahasa saja (tidak bilingual), operasi symmetric difference antara set pengguna bahasa Inggris dan set pengguna bahasa Korea akan memberikan jawaban tersebut.

Secara matematis, symmetric difference antara set A dan B dapat pula dinyatakan sebagai gabungan (union) dari kedua set dikurangi irisan (intersection) keduanya. Hal ini menunjukkan bahwa symmetric difference dapat pula dicapai dengan mengombinasikan operasi union dan intersection dalam satu ekspresi.

Keempat operasi set tersebut sangat berguna dalam berbagai situasi pemrograman nyata, terutama ketika bekerja dengan data yang perlu dibandingkan, digabungkan, atau difilter berdasarkan keanggotaannya. Dengan memanfaatkan operasi-operasi set ini, banyak permasalahan yang berkaitan dengan pengelompokan dan perbandingan data dapat diselesaikan dengan kode yang lebih ringkas dan efisien.

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github: https://github.com/gathasja/71251218_Agatha.git

SOAL 1

```
1 n = int(input('Masukkan jumlah kategori: '))
2
3 data_aplikasi = {}
4
5 for i in range(n):
6     nama_kategori = input('Masukkan nama kategori: ')
7     print('Masukkan 5 nama aplikasi di kategori', nama_kategori)
8     aplikasi = set()
9     for j in range(5):
10        nama_aplikasi = input('Nama aplikasi: ')
11        aplikasi.add(nama_aplikasi)
12        data_aplikasi[nama_kategori] = aplikasi
13
14 for kategori, apps in data_aplikasi.items():
15     print(f'{kategori}: {apps}')
16
17 semua_set = list(data_aplikasi.values())
18
19 muncul_semua = semua_set[0]
20 for s in semua_set[1:]:
21     muncul_semua = muncul_semua & s
22 print(f'\naplikasi yang muncul di semua kategori: {muncul_semua}')
23
24 print("\naplikasi yang hanya muncul di 1 kategori:")
25 for kategori, apps in data_aplikasi.items():
26     set_lain = set()
27     for k, v in data_aplikasi.items():
28         if k != kategori:
29             set_lain = set_lain | v
30     hanya_disini = apps - set_lain
31     print(f'{kategori}: {hanya_disini}')
32
33 if n > 2:
34     print("\naplikasi muncul di 2 kategori:")
35     kategori_list = list(data_aplikasi.keys())
36     tepat_dua = set()
37     for i in range(len(kategori_list)):
38         for j in range(i + 1, len(kategori_list)):
39             irisan_dua = data_aplikasi[kategori_list[i]] & data_aplikasi[kategori_list[j]]
40             for k in range(len(kategori_list)):
41                 if k != i and k != j:
42                     irisan_dua = irisan_dua - data_aplikasi[kategori_list[k]]
43             tepat_dua = tepat_dua | irisan_dua
44     print(f'{tepat_dua}')
```

Penjelasan soal01:

1. `n = int(input('Masukkan jumlah kategori: '))` -> meminta user untuk mengisi jumlah kategori.
2. `data_aplikasi = {}` -> membuat dictionary kosong untuk menyimpan data.
3. Program meminta input jumlah kategori (n), lalu untuk setiap kategori membuat set kosong, meminta 5 nama aplikasi dan memasukkannya ke dalam set menggunakan `add()`, kemudian menyimpannya ke dalam dictionary dengan key berupa nama kategori.
4. Loop semua kategori dan cetak isinya.

```
for kategori, apps in data_aplikasi.items():
    print(f'{kategori}: {apps}')
    nama_kategori = input('Masukkan nama kategori: ')
    print('Masukkan 5 nama aplikasi di kategori', nama_kategori)
    aplikasi = set()
    for j in range(5):
        nama_aplikasi = input('Nama aplikasi: ')
        aplikasi.add(nama_aplikasi)
    data_aplikasi[nama_kategori] = aplikasi
```

5. Semua set aplikasi dari dictionary diambil, kemudian dimulai dengan set pertama dan diintersection (&) secara berurutan dengan set berikutnya untuk mendapatkan aplikasi yang muncul di semua kategori.
6. Untuk setiap kategori, program menggabungkan semua aplikasi dari kategori lain ke dalam set_lain, lalu menghitung apps dikurangi (-) set_lain untuk mendapatkan aplikasi yang hanya muncul di kategori tersebut dan tidak di kategori lain.

```

semua_set = list(data_aplikasi.values())
muncul_semua = semua_set[0]
for s in semua_set[1:]:
    muncul_semua = muncul_semua & s
print(f"\naplikasi yang muncul di semua kategori: {muncul_semua}")
print("\naplikasi yang hanya muncul di 1 kategori:")
for kategori, apps in data_aplikasi.items():
    set_lain = set()
    for k, v in data_aplikasi.items():
        if k != kategori:
            set_lain = set_lain | v
    hanya_disini = apps - set_lain
    print(f"{kategori}: {hanya_disini}")

```

7. Jika $n > 2$, program akan memeriksa setiap pasangan kategori (i, j), mengambil irisan (&) aplikasi dari kedua kategori, lalu mengurangi (-) aplikasi yang juga muncul di kategori ketiga (k), kemudian menggabungkan (union) hasil akhir ke dalam himpunan tepat_dua.

```

if n > 2:
    print("\naplikasi muncul di 2 kategori:")
    kategori_list = list(data_aplikasi.keys())
    tepat_dua = set()
    for i in range(len(kategori_list)):
        for j in range(i + 1, len(kategori_list)):
            irisan_dua = data_aplikasi[kategori_list[i]] & data_aplikasi[kategori_list[j]]
            for k in range(len(kategori_list)):
                if k != i and k != j:
                    irisan_dua = irisan_dua - data_aplikasi[kategori_list[k]]
            tepat_dua = tepat_dua | irisan_dua
    print(f"{tepat_dua}")

```


SOAL 2

```
1 list_a = [10, 20, 30, 20, 10, 40]
2 print(f"list      : {list_a}")
3 print(f"list ke Set: {set(list_a)}")
4 print()
5
6 set_a = {'apel', 'jeruk', 'mangga', 'apel'}
7 print(f"set      : {set_a}")
8 print(f"set ke list: {list(set_a)}")
9 print()
10
11 tuple_a = (5, 10, 15, 10, 5, 20)
12 print(f"tuple     : {tuple_a}")
13 print(f"tuple ke set: {set(tuple_a)}")
14 print()
15
16 set_b = {'merah', 'biru', 'hijau', 'merah'}
17 print(f"set      : {set_b}")
18 print(f"set ke tuple: {tuple(set_b)}")
```

Penjelasan soal02:

1. `list_a = [10, 20, 30, 20, 10, 40]` -> membuat List bernama `list_a` berisi angka-angka.
2. `print(f"list : {list_a}")` -> menampilkan isi dari list.
3. `print(f"list ke Set: {set(list_a)}")` -> mengonversi list ke set, duplikat dari angka kembar akan dihapus (hanya ditampilkan sekali).
4. `print()`
5. `set_a = {'apel', 'jeruk', 'mangga', 'apel'}` -> membuat Set bernama `set_a`, nilai 'apel' yang kedua otomatis diabaikan.
6. `print(f"set : {set_a}")` -> menampilkan isi dari set.
7. `print(f"set ke list: {list(set_a)}")` -> mengonversi set ke list.
8. `print()`
9. `tuple_a = (5, 10, 15, 10, 5, 20)` -> membuat tuple bernama `tuple_a`, boleh ada item duplikat.
10. `print(f"tuple : {tuple_a}")` -> menampilkan isi tuple.
11. `print(f"tuple ke set: {set(tuple_a)}")` -> mengonversi tuple ke set, angka yang memiliki duplikat otomatis hanya ditulis sekali.
12. `print()`
13. `set_b = {'merah', 'biru', 'hijau', 'merah'}` -> membuat set bernama `set_b`.
14. `print(f"set : {set_b}")` -> menampilkan isi set.
15. `print(f"set ke tuple: {tuple(set_b)}")` -> mengonversi set ke tuple.

SOAL 3

```
import string
def baca_kata(nama_file):
    try:
        with open(nama_file, 'r') as f:
            isi = f.read()
    except FileNotFoundError:
        print(f"Error: File '{nama_file}' tidak ditemukan!")
        exit()
    except Exception as e:
        print(f"Error: File '{nama_file}' tidak bisa dibaca! ({e})")
        exit()

    isi = isi.lower()
    for tanda in string.punctuation:
        isi = isi.replace(tanda, ' ')
    return set(isi.split())

file1 = input("Masukkan nama file pertama : ")
file2 = input("Masukkan nama file kedua : ")

set1 = baca_kata(file1)
set2 = baca_kata(file2)
kata_sama = set1 & set2

print(f"\njumlah kata unik di {file1} : {len(set1)} kata")
print(f"jumlah kata unik di {file2} : {len(set2)} kata")
print(f"jumlah kata yang muncul di 2 file tsb : {len(kata_sama)} kata")

print(f"\nkata yang muncul di 2 file tersebut:")
for kata in sorted(kata_sama):
    print(f"{kata}")
```

Penjelasan soal03:

1. Mengimpor modul string untuk mengakses karakter tanda baca.

```
import string
```

2. Fungsi untuk membaca file dan mengembalikan set kata unik

```
def baca_kata(nama_file):
    .
```

3. Membuka file dalam mode baca. Jika file tidak ditemukan atau error lain, tampilkan pesan dan keluar program.

```
try:
    with open(nama_file, 'r') as f:
        isi = f.read()
except FileNotFoundError:
    print(f"Error: File '{nama_file}' tidak ditemukan!")
    exit()
except Exception as e:
    print(f"Error: File '{nama_file}' tidak bisa dibaca! ({e})")
    exit()
```

4. Mengubah semua huruf menjadi huruf kecil agar tidak case-sensitive.

```
isi = isi.lower()
```

5. Loop semua karakter tanda baca dan ganti dengan spasi. `split()` memisahkan teks menjadi list kata, lalu `set()` menghilangkan duplikasi.

```
for tanda in string.punctuation:
    isi = isi.replace(tanda, ' ')
return set(isi.split())
```

6. Meminta inputan dari user.

```
file1 = input("Masukkan nama file pertama : ")
file2 = input("Masukkan nama file kedua : ")
```

7. Memanggil fungsi `baca_kata()` untuk mengolah kedua file, lalu Operator `&` (intersection) mencari kata yang muncul di kedua file.

```
set1 = baca_kata(file1)
set2 = baca_kata(file2)
kata_sama = set1 & set2
```

8. Menampilkan jumlah kata unik masing-masing file dan jumlah kata yang sama.

```
print(f"\njumlah kata unik di {file1} : {len(set1)} kata")
print(f"jumlah kata unik di {file2} : {len(set2)} kata")
print(f"jumlah kata yang muncul di 2 file tsb : {len(kata_sama)} kata")
```

9. `sorted()` mengurutkan kata secara alfabetis, lalu ditampilkan satu per satu.

```
print(f"\nkata yang muncul di 2 file tersebut:")
for kata in sorted(kata_sama):
    print(f"{kata}")
```